Musterlösung zu Übung 1

WWW HTTP URI HTML

Einführung in Webengineering · WS 2026/27

Vergleichen Sie Ihre Antworten mit den folgenden Lösungsvorschlägen. Andere fachlich korrekte Formulierungen und HTML-Umsetzungen sind ebenfalls möglich.

01 Internet und World Wide Web unterscheiden

a) Internet und WWW: Das Internet ist ein weltweiter Verbund miteinander vernetzter Rechnernetze. Das WWW ist ein darauf aufbauender Dienst aus verknüpften Ressourcen, die typischerweise über HTTP abgerufen werden. E-Mail über SMTP ist ein Internetdienst außerhalb des WWW; Webmail ist dagegen eine Weboberfläche für E-Mail.

b) Rollen und Technologien: Der Browser fordert Ressourcen an und stellt etwa HTML-Dokumente dar. Der Webserver nimmt Anfragen entgegen und liefert Antworten. HTTP regelt die Kommunikation, eine URI identifiziert die Ressource, und HTML beschreibt Struktur und Bedeutung der Inhalte.

c) Dezentrale Verlinkung: Verweise können unabhängig und ohne zentrale Linkverwaltung gesetzt werden. Wird ein Ziel verschoben oder gelöscht, wird der Verweis jedoch nicht automatisch angepasst: Es kann ein „toter Link" entstehen.

02 Eine Webadresse zerlegen

a) Bestandteile: Die Beispieladresse besteht aus folgenden Teilen:

Bestandteil	Wert
Schema	`https`
Host	`lernen.example.org`
Port	`8443`
Pfad	`/we/kapitel01.html`
Query	`modus=uebung&sprache=de`
Fragment	`html`
Authority	`lernen.example.org:8443`

b) Fragment: „`html`" bezeichnet hier ein Sprungziel, zum Beispiel ein Element mit `id="html"`. Der Browser wertet das Fragment aus; es gehört nicht zum HTTP-Anfrageziel und wird beim Abruf nicht an den Server übertragen.

c) URI und URL: Eine URI identifiziert eine Ressource. Eine URL ist eine URI, die außerdem einen Zugriffsweg beschreibt. Die HTTPS-Adresse ist damit sowohl URI als auch URL.

Nachlesen: RFC 3986 — URI-Syntax und Bestandteile

03 Relative Verweise auflösen

Basis ist die Dokumentadresse `https://lernen.example.org/we/kapitel01/index.html`.

a) Verweis	Absolutes Ziel
`slide02.html`	`https://lernen.example.org/we/kapitel01/slide02.html`
`../material.pdf`	`https://lernen.example.org/we/material.pdf`
`/kontakt.html`	`https://lernen.example.org/kontakt.html`
`#aufgabe7`	`https://lernen.example.org/we/kapitel01/index.html#aufgabe7`

b) Unterschied der Pfade: „`../material.pdf`“ geht vom aktuellen Dokumentverzeichnis eine Ebene nach oben, hier nach `/we/`. „`/material.pdf`“ beginnt dagegen direkt an der Wurzel des Hosts; das Ziel ist `https://lernen.example.org/material.pdf`.

c) Verschieben der Website: „`slide02.html`“ und „`#aufgabe7`“ bleiben sinnvoll, wenn die Zieldatei mit umzieht beziehungsweise die Ziel-ID erhalten bleibt. „`../material.pdf`“ bleibt korrekt, wenn auch diese Datei mit unveränderter relativer Lage verschoben wird. Beim Verschieben nur von `kapitel01` kann dieser Verweis brechen.

„`/kontakt.html`“ bleibt auf die Wurzel des Hosts bezogen. Zieht die Kontaktseite in ein neues Unterverzeichnis, folgt dieser Verweis nicht mit. Er muss an den neuen Pfad angepasst werden.

04 Anfrage und Antwort lesen

a) Anfrage: `GET` ist die Methode. Das Anfrageziel lautet `/we/kapitel01.html?modus=uebung`; es enthält Pfad und Query. `HTTP/1.1` ist die Protokollversion. „`Host: lernen.example.org`“ nennt den angesprochenen Host, damit ein Server beispielsweise mehrere Websites unterscheiden kann. Bei einem abweichenden Port würde auch dieser angegeben.

b) Antwort: „`HTTP/1.1 200 OK`“ ist die Statuszeile: Version, numerischer Statuscode und Statusbeschreibung. `200` meldet eine erfolgreiche Bearbeitung. Der Header „`Content-Type: text/html; charset=utf-8`“ kennzeichnet den Body als HTML mit `UTF-8`-Zeichenkodierung.

Nach der Leerzeile beginnt der Body mit `<!doctype html>`. Dort steht das eigentliche HTML-Dokument. „`Accept: text/html`“ in der Anfrage nennt dagegen HTML als gewünschtes Antwortformat.

c) Zustandslosigkeit: HTTP behandelt Anfragen grundsätzlich unabhängig voneinander; eine Anfrage baut nicht automatisch auf einem zuvor gespeicherten Dialogzustand auf. Anmeldungen oder Warenkörbe lassen sich auf Anwendungsebene über Sitzungen abbilden, etwa mit einer Sitzungskennung in einem Cookie.

Eine bestehende Transportverbindung darf für mehrere Anfragen genutzt werden. Zustandslosigkeit verlangt daher keinen neuen Verbindungsaufbau pro Anfrage; Verbindungswiederverwendung und Anwendungszustand sind verschiedene Dinge.

Nachlesen: MDN — Überblick über HTTP

05 Methoden und Statuscodes auswählen

a) Passende Methoden: Die Methode beschreibt die beabsichtigte Aktion auf der Ressource.

Fall	Methode	Begründung
Ressource abrufen	`GET`	Fordert eine Darstellung der Ressource an.
Nur Antwort-Header abrufen	`HEAD`	Entspricht `GET`, überträgt aber keinen Antwort-Body.
Formular verarbeiten lassen	`POST`	Übermittelt Daten zur Verarbeitung am Ziel.
Ressource vollständig ersetzen	`PUT`	Legt die neue Darstellung an der bekannten Ziel-URI fest.
Ressource löschen	`DELETE`	Fordert das Entfernen der Zielressource an.

b) Safe und idempotent: „Safe" bedeutet, dass keine Änderung am Serverzustand angefordert wird. „Idempotent" bedeutet, dass mehrere identische Anfragen dieselbe beabsichtigte Wirkung haben wie eine einzelne Anfrage.

Methode	Safe	Idempotent	Einordnung
`GET`	Ja	Ja	Liest die Ressource.
`POST`	Nein	Nein	Kann bei Wiederholung erneut etwas auslösen.
`PUT`	Nein	Ja	Setzt die Zielressource auf denselben Zustand.
`DELETE`	Nein	Ja	Nach dem Löschen bleibt die Ressource entfernt.

Idempotenz betrifft die beabsichtigte Wirkung, nicht identische Antworten. Ein erstes `DELETE` kann beispielsweise `204` liefern, ein zweites `404`. Protokollierung ist auch bei `GET` möglich. Eine Anwendung kann `POST` gegen Doppelverarbeitung absichern; die Methode selbst garantiert dies nicht.

c) Statuscodes: Die erste Ziffer bestimmt die Statusklasse.

Code	Klasse	Passender Anwendungsfall
`200 OK`	`2xx` Erfolg	Eine angeforderte HTML-Seite wird mit Body geliefert.
`204 No Content`	`2xx` Erfolg	Eine Löschung gelingt; es gibt keinen Antwortinhalt.
`302 Found`	`3xx` Weiterleitung	Vorübergehende Weiterleitung, Ziel im `Location`-Header.
`403 Forbidden`	`4xx` Clientfehler	Der Server versteht die Anfrage, verweigert aber den Zugriff.
`404 Not Found`	`4xx` Clientfehler	Die angeforderte Ressource wird nicht gefunden.
`500 Internal Server Error`	`5xx` Serverfehler	Ein unerwarteter Fehler verhindert die Bearbeitung.

Nachlesen: MDN — HTTP-Methoden und ihre Eigenschaften

Nachlesen: MDN — HTTP-Statuscodes

06 HTTP im Browser beobachten

a–c) Beispielprotokoll: Die Messwerte hängen von der gewählten Website, dem Browser und dem Cache ab. Die folgende Tabelle ist ein fiktives Beispiel für die Dokumentation; sie stellt keine tatsächlich gemessenen Anfragen dar.

Merkmal	HTML-Hauptdokument	Zusätzliche Ressource
URL	`https://lernen.example.org/`	`https://lernen.example.org/bilder/logo.png`
Methode	`GET`	`GET`
Statuscode	`200`	`200`
`Content-Type`	`text/html; charset=utf-8`	`image/png`
HTTP-Version	`h2` (`HTTP/2`)	`h2` (`HTTP/2`)

Das HTML-Hauptdokument enthält Verweise auf weitere Ressourcen. Für ein externes Bild, eine CSS-Datei oder ein Skript kann der Browser eine weitere Anfrage senden. `Content-Type` beschreibt jeweils den gelieferten Medientyp; nicht jede Ressource ist HTML.

Weiterleitung: Eine Antwort wie `301` oder `302` enthält gewöhnlich einen `Location`-Header. Der Browser ruft das dort angegebene Ziel in einer weiteren Anfrage auf. Im Protokoll können deshalb mehrere Dokumentanfragen zum selben Seitenaufruf stehen.

Cache: Eine noch frische gespeicherte Ressource kann ohne Serveranfrage genutzt werden. Muss der Browser sie prüfen, kann er eine bedingte Anfrage senden. Eine Antwort `304 Not Modified` bestätigt dann die gespeicherte Fassung, ohne deren Inhalt erneut zu übertragen.

Ihre Lösung ist vollständig, wenn sie beide Ressourcen mit den geforderten Angaben dokumentiert und die tatsächlich sichtbaren Besonderheiten erklärt. Zeigt Ihr Browser die HTTP-Version nicht an, vermerken Sie dies statt eine Version zu raten.

Nachlesen: MDN — HTTP-Caching

07 Eine semantische HTML-Seite erstellen

a–c) Beispielumsetzung: Die nächste Seite zeigt eine vollständige `index.html`. Sie erfüllt Aufgabe 7 und enthält bereits die Links, das Bild und den Kontaktabschnitt aus Aufgabe 8. Die Themenliste ist ungeordnet, der Ablauf ist als Reihenfolge nummeriert.

d) `title` und `h1`: `title` steht im `head` und benennt das Dokument, etwa für den Browser-Tab oder ein Lesezeichen. `h1` steht im `body` und ist die sichtbare Hauptüberschrift der Seite. Beide dürfen ähnlich formuliert sein, haben aber unterschiedliche Aufgaben.

Semantische Elemente: `header`, `nav`, `main`, `section` und `footer` kennzeichnen die Rolle ihrer Inhalte. Ein `div` gruppiert Elemente ohne eine solche besondere Bedeutung. `strong` markiert Wichtigkeit, `em` eine inhaltliche Betonung; die standardmäßige Fettschrift oder Kursivschrift ist nur deren Darstellung.

07 Musterdatei `index.html`

Die beiden Lernabschnitte haben eigene Unterüberschriften. Der zusätzliche Kontaktabschnitt ist das Sprungziel aus Aufgabe 8.

`kapitel01/index.html`

```
<!doctype html>
<html lang="de">
<head>
  <meta charset="utf-8">
  <title>Lerngruppe Web Engineering</title>
</head>
<body>
  <header><h1>Unsere Lerngruppe Web Engineering</h1></header>
  <nav aria-label="Hauptnavigation">
    <a href="material.html">Lernmaterialien</a>
    <a href="#kontakt">Kontakt zur Lerngruppe</a>
    <a href="https://developer.mozilla.org/de/docs/Web/HTML">
      HTML-Dokumentation bei MDN</a>
  </nav>
  <main>
    <section>
      <h2>Unsere Themen</h2>
      <p>Wir erarbeiten die Grundlagen gemeinsam.
        <strong>Bringen Sie Ihre Fragen mit.</strong></p>
      <ul><li>WWW und URI</li><li>HTTP</li><li>HTML</li></ul>
      <img src="bilder/lerngruppe.png"
           alt="Drei Studierende arbeiten an einem HTML-Dokument.">
    </section>
    <section>
      <h2>Ablauf eines Treffens</h2>
      <p>Wir erklären Lösungswege <em>in eigenen Worten</em>.</p>
      <ol>
        <li>Offene Fragen sammeln</li>
        <li>Aufgaben gemeinsam bearbeiten</li>
        <li>Ergebnisse vergleichen</li>
      </ol>
    </section>
    <section id="kontakt">
      <h2>Kontakt</h2>
      <p>Die Lerngruppe ist über das Forum des Kurses erreichbar.</p>
    </section>
  </main>
  <footer><p>Lerngruppe Webengineering</p></footer>
</body>
</html>
```

Der Bildinhalt ist beispielhaft angenommen. Verwenden Sie ein eigenes oder frei verwendbares Bild und passen Sie den `alt`-Text an dessen tatsächlichen Inhalt an.

Nachlesen: HTML Living Standard — semantische Abschnitte

08 Links und Bilder ergänzen

a) Dateistruktur und zweite Seite: `index.html` und `material.html` liegen im selben Verzeichnis. Daher genügen die Dateinamen für die gegenseitigen Verweise.

```
kapitel01/
  index.html
  material.html
  bilder/
    lerngruppe.png
```

`kapitel01/material.html`

```
<!doctype html>
<html lang="de">
<head>
  <meta charset="utf-8">
  <title>Lernmaterialien zur Lerngruppe</title>
</head>
<body>
  <header><h1>Lernmaterialien</h1></header>
  <nav aria-label="Hauptnavigation">
    <a href="index.html">Startseite der Lerngruppe</a>
  </nav>
  <main>
    <h2>Grundlagen wiederholen</h2>
    <p>Nutzen Sie das erste Kapitel des Slidecasts zu
      WWW, HTTP, URI und HTML.</p>
  </main>
  <footer><p>Lerngruppe Webengineering</p></footer>
</body>
</html>
```

b) Sprungziel und externer Link: `href="#kontakt"` verweist auf das Element mit `id="kontakt"` in `index.html`. Die ID muss innerhalb dieses Dokuments eindeutig sein. Die absolute HTTPS-Adresse des MDN-Links führt zur externen HTML-Dokumentation.

c) Bild und Textalternative: `src="bilder/lerngruppe.png"` verweist in den Unterordner `bilder`. Der `alt`-Text vermittelt den relevanten Bildinhalt, etwa für Screenreader oder wenn das Bild nicht geladen wird. Für das angenommene Gruppenfoto beschreibt er Personen und Tätigkeit; „Bild“ oder der Dateiname wäre wenig hilfreich.

d) Erwartetes Testergebnis: Prüfen Sie den Hin- und Rückweg zwischen den HTML-Dateien, den Sprung zu Kontakt, den externen Dokumentationslink und die Bildanzeige. Nach dem Verschieben des gesamten Ordners funktionieren die lokalen Verweise weiter, weil die Dateien ihre relative Lage zueinander behalten. Der absolute externe Link ist vom Speicherort unabhängig.

Nachlesen: HTML Living Standard — Bilder und Textalternativen

09 Fehler finden und Darstellung erklären

a) Korrigierter Ausschnitt: Eine mögliche Korrektur lautet:

```
<h1>Unsere Lerngruppe</h1>
<p>Wir lernen gemeinsam.</p>
<ul>
  <li>HTTP</li>
  <li>HTML</li>
</ul>
<a href="material.html">Materialien der Lerngruppe</a>
<img src="bilder/lerngruppe.png"
     alt="Drei Studierende arbeiten an einem HTML-Dokument.">
```

Fehler im Original	Korrektur und Begründung
`<h1>` … `</h2>`	Mit `</h1>` schließen; Start- und End-Tag müssen zusammenpassen.
`<ul>` innerhalb von `<p>`	Den Absatz vor der Liste beenden. Eine Liste ist kein zulässiger Inhalt eines `p`-Elements.
`</a` ohne abschließendes `>`	End-Tag als `</a>` vervollständigen.
`scr` statt `src`	Das richtige Attribut für die Bildadresse lautet `src`.
Fehlende Textalternative	`alt` ergänzen und an den tatsächlichen Bildinhalt anpassen.

b) Absatz und Liste im DOM: Beim Start-Tag `<ul>` beendet der HTML-Parser das offene `p`-Element automatisch. Die Liste wird im DOM deshalb ein Geschwisterelement des Absatzes, kein Kind. Das danach allein stehende `</p>` erzeugt hier außerdem ein zusätzliches leeres `p`-Element. Im vollständigen Testdokument ergibt sich für diesen Teil:

```
<p>Wir lernen gemeinsam.</p>
<ul><li>HTTP</li><li>HTML</li></ul>
<p></p>
```

Die Elementansicht zeigt die vom Browser aufgebaute Struktur. Die Quelltextansicht zeigt weiterhin die fehlerhafte Eingabe. Browser reparieren viele HTML-Fehler nach festgelegten Regeln; dadurch können sich beide Ansichten unterscheiden.

c) `div` und `span`: Ohne eigenes CSS erzeugt `div` gewöhnlich einen Block mit einer neuen Zeile davor und danach. `span` bleibt gewöhnlich im laufenden Text. CSS kann dieses Darstellungsverhalten ändern.

Eine sichtbare Ausgabe belegt keine Fehlerfreiheit. Der Browser kann ungültiges HTML reparieren und trotzdem etwas anzeigen. Zur Prüfung gehören daher Quelltext, DOM und die Regeln für die zulässige Verschachtelung; ein HTML-Validator kann dabei helfen.

Nachlesen: HTML Living Standard — Regeln für den HTML-Parser

10 Ein Formular und seine Anfrage untersuchen

a) Vollständiges Suchformular: Die folgende Datei verwendet `GET`. Für Teil c wird nur `method="get"` durch `method="post"` ersetzt.

`kapitel01/suche.html`

```
<!doctype html>
<html lang="de">
<head>
  <meta charset="utf-8">
  <title>Themensuche zur Lerngruppe</title>
</head>
<body>
  <main>
    <h1>Themensuche</h1>
    <form action="https://example.org/suche" method="get">
      <p><label for="suchbegriff">Suchbegriff</label>
        <input type="text" id="suchbegriff" name="q"></p>
      <p><label for="thema">Thema</label>
        <select id="thema" name="thema">
          <option value="http">HTTP</option>
          <option value="html">HTML</option>
        </select></p>
      <button type="submit">Suchen</button>
    </form>
  </main>
</body>
</html>
```

b) `GET`-Anfrage: Für „`HTML lernen`“ und die Auswahl `html` werden die Daten als `q=HTML+lernen&thema=html` kodiert. Das Pluszeichen steht in dieser Formularkodierung für ein Leerzeichen. Daraus entsteht folgende Zieladresse:

```
https://example.org/suche?q=HTML+lernen&thema=html
```

c) `POST`-Anfrage: Die Methode lautet nun `POST`. Die Zieladresse ist `https://example.org/suche`; die Formulardaten stehen im Anfrage-Body. Bei diesem Formular lautet der `Content-Type` `application/x-www-form-urlencoded` und der Body weiterhin `q=HTML+lernen&thema=html`. Die Antwort des Beispielservers ist für den Vergleich unerheblich.

d) `name` und `id`: `name` legt den Schlüssel beim Absenden fest; `id` identifiziert das Element im Dokument und verbindet es über `for` mit dem `label`. Ohne `name` am Textfeld wird dessen Wert nicht übertragen. Hier bleibt dann nur `thema=html`.

Vertraulichkeit: `POST` allein verschlüsselt nichts. Vertraulichkeit auf dem Übertragungsweg bietet HTTPS – bei `GET` ebenso wie bei `POST`. `POST`-Daten stehen im Body statt in der Adresszeile; für den Zielserver sind sie weiterhin lesbar.

Nachlesen: MDN — Formulardaten senden und empfangen